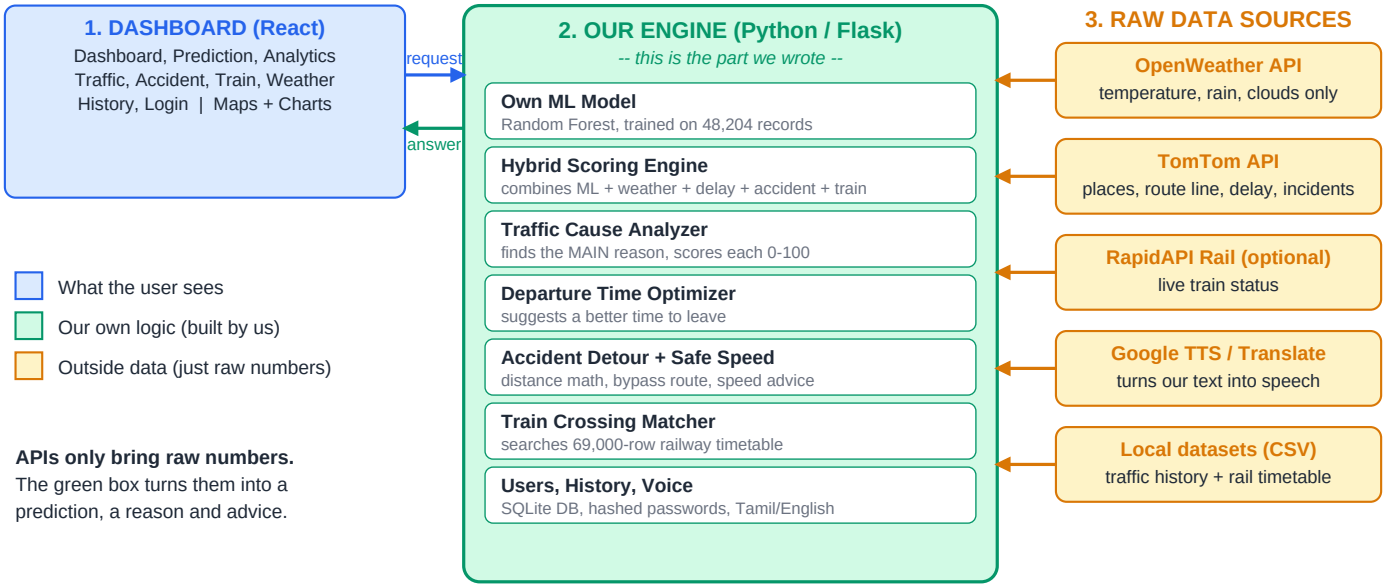
Smart Traffic Prediction System

What we built, how it works, and why it is more than "just APIs" | Project overview for staff

The short answer. An API is like a thermometer: it only gives a raw number ("it is raining", "the road is 12 minutes slow"). It does **not** say whether traffic will be heavy at 6 PM tomorrow, **why**, or **when you should leave**. Our system does that. We **trained our own machine-learning model** on 48,204 hours of real traffic data, wrote a **scoring engine** that combines five factors, a **cause analyzer** that explains the result, a **train-crossing checker** over a 69,000-row railway timetable, accident detour + safe-speed logic, user accounts, history and Tamil/English voice alerts. The APIs are only the raw ingredients; **the recipe is ours.**

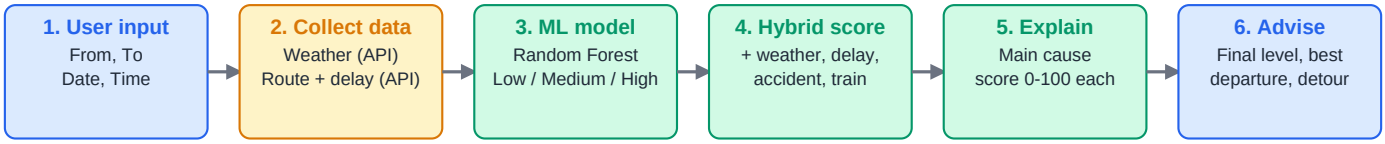
1. System architecture (three layers)



2. Module by module: what the API gives vs. what we built

Module	API gives only	Our own work on top
Traffic Prediction	Weather numbers; route travel time	Own trained ML model + hybrid scoring + final Low/Moderate/High decision
Cause Analysis	Nothing	Scores 5 causes (history, weather, delay, accident, train) 0-100, picks the main one, avoids double counting, writes a plain-English reason
Departure Advice	Nothing	Suggests leaving 20-45 min earlier, or later if it is too close to now
Accident & Safety	List of incidents in a map area	Checks which incidents are really on *your* route (Haversine distance), safe speed by distance, generates a local bypass detour
Train Crossing	Live status (optional)	Matches source/destination stations in the Indian Railways timetable (69k rows), filters trains within +/- 2 h of your trip
Analytics & Live View	Nothing	Hourly traffic curve and peak hour built from the historical dataset; charts and KPIs
Users & History	Nothing	Register/login (SHA-256 hashed passwords), saved search history in SQLite
Voice Alerts	Text-to-speech engine	We decide what to say and when; Tamil translation before speaking

3. How one prediction is made (step by step)

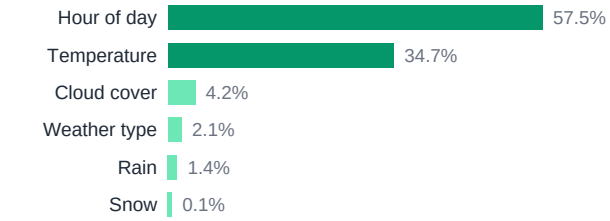


Example: 6 PM + rain + 12 min TomTom delay -> ML says "High" (+2), rain (+1), delay (+1) = score 4
-> Final: HIGH traffic. Main cause: traffic delay. Advice: "leave around 17:15 to reduce delay".

4. Our machine-learning model

Data: Metro Interstate Traffic Volume dataset, **48,204 hourly records** (traffic count + weather for each hour).
Labels we created: under 2,000 vehicles/hour = Low, 2,000-5,000 = Medium, above 5,000 = High.
Inputs (6 features): hour, temperature, rain, snow, cloud cover, weather type.
Algorithm: Random Forest classifier (100 decision trees, scikit-learn). 80% of data for training, 20% kept hidden for testing.
Result: 76.2% accuracy on the hidden test data (3 classes; random guessing would be about 33%).
Saved model: `model.pkl`, loaded once when the server starts, answers in milliseconds.

What the model learned matters most



Time of day drives traffic most - rush hours - which matches real life.

5. Hybrid scoring engine: how the final answer is decided

Factor	Source	Rule we wrote	Points
ML prediction	Our model	High / Medium / Low	+2 / +1 / 0
Bad weather	OpenWeather	Rain, drizzle, snow or thunderstorm	+1
Road delay	TomTom	Delay more than 10 minutes	+1
Accident / hazard	TomTom + our route check	Accident on route / other hazard	+4 / +2
Train crossing	Railway timetable	A train overlaps your travel time	+1

Total 4 or more = HIGH

Total 2-3 = MODERATE

Total 0-1 = LOW

Why hybrid? The ML model knows *normal* patterns; the live factors catch *today's* surprises (an accident, a storm). Together they are more reliable than either alone.

6. Technology used

Layer	Technology	Layer	Technology
Frontend	React 19, Vite, React Router	**Machine learning**	scikit-learn, pandas, NumPy, joblib
Maps & charts	Leaflet maps, Chart.js	**Database**	SQLite (users, search history)
Backend	Python, Flask REST API (16 endpoints)	**Voice**	gTTS + Google Translate (Tamil)

7. Honest status and next steps

• The **Analytics / Live** charts are currently built from historical hourly averages with small time-based variation (a simulation), not from live sensors. • Journey **temporary-stop / recalculate** endpoints are placeholders. • Planned: retrain the model on local Indian city traffic data, add model evaluation dashboard, store live predictions to improve the model over time.